

**EXERCISE 10.12**

Implement `Foldable[List]`, `Foldable[IndexedSeq]`, and `Foldable[Stream]`. Remember that `foldRight`, `foldLeft`, and `foldMap` can all be implemented in terms of each other, but that might not be the most efficient implementation.

**EXERCISE 10.13**

Recall the binary `Tree` data type from chapter 3. Implement a `Foldable` instance for it.

```
sealed trait Tree[+A]
case object Leaf[A](value: A) extends Tree[A]
case class Branch[A](left: Tree[A], right: Tree[A]) extends Tree[A]
```

**EXERCISE 10.14**

Write a `Foldable[Option]` instance.

**EXERCISE 10.15**

Any `Foldable` structure can be turned into a `List`. Write this conversion in a generic way:

```
def toList[A](fa: F[A]): List[A]
```

10.6 Composing monoids

The `Monoid` abstraction in itself is not all that compelling, and with the generalized `foldMap` it's only slightly more interesting. The real power of monoids comes from the fact that they *compose*.

This means, for example, that if types `A` and `B` are monoids, then the tuple type `(A, B)` is also a monoid (called their *product*).

**EXERCISE 10.16**

Prove it. Notice that your implementation of `op` is obviously associative so long as `A.op` and `B.op` are both associative.

```
def productMonoid[A,B](A: Monoid[A], B: Monoid[B]): Monoid[(A,B)]
```